台鋼新零售全端工程師筆試｜情境六～九

> 本稿依題目提供的條件作答，沿用前幾題的票務領域。未明確指定的實作細節會標成設計假設；類別圖用來表達角色、責任與依賴，不代表唯一實作方式。

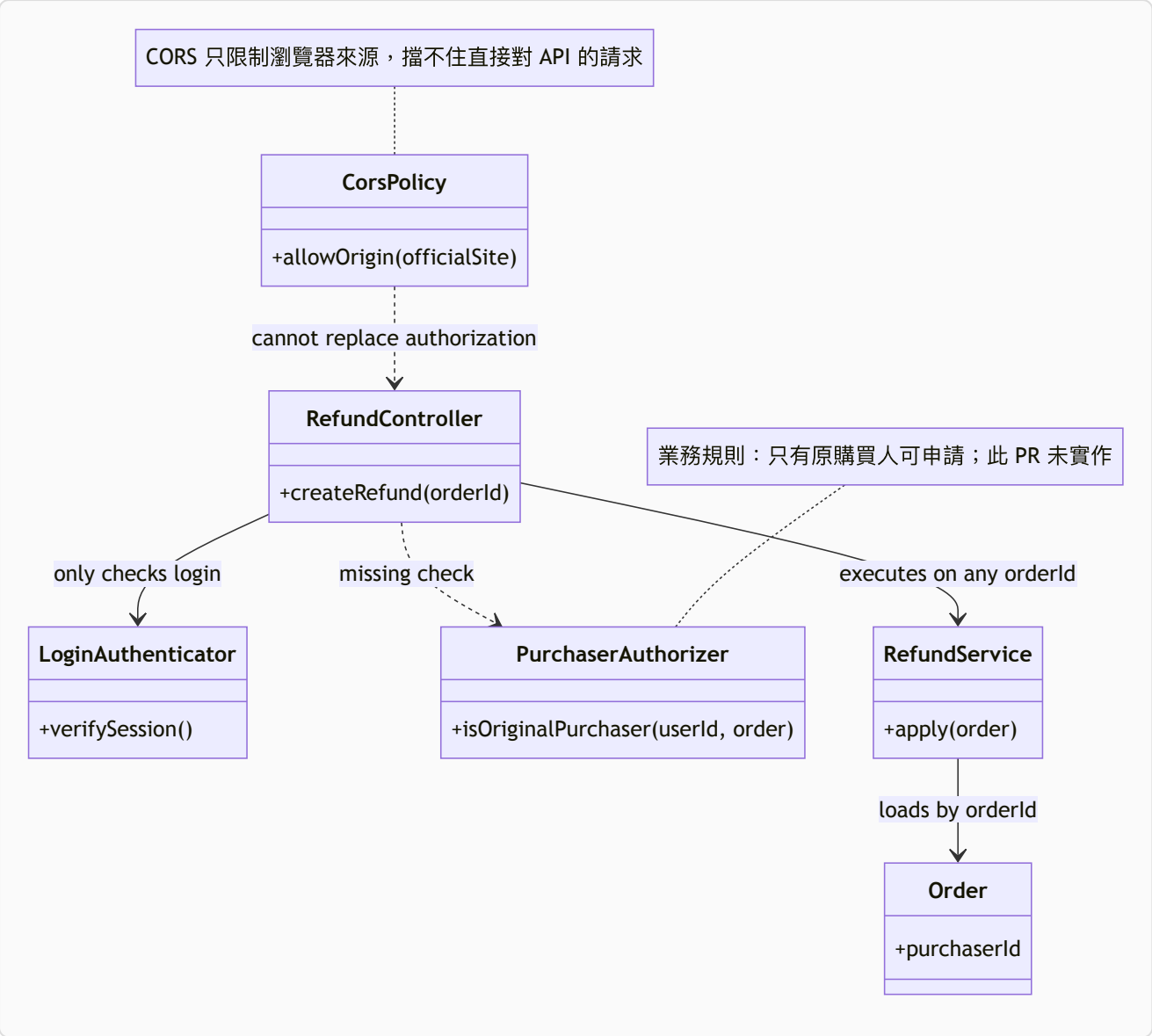
情境六｜登入了，就能操作嗎？

第 1 題：會讓這份 PR 通過嗎？最需要先處理的兩個問題

不會通過，我會 Request changes。最需要先處理的兩個問題：

PR 中的做法	問題	可能後果
API 只驗證「已登入」，依 orderId 取得訂單後就執行，未比對原購買人	缺少操作授權：登入只證明「你是會員」，不代表「你有權操作這筆訂單」（IDOR 類缺陷）	任何登入會員只要知道或猜到 orderId，就能替別人的訂單申請退款，直接造成金錢損失與客訴
用 `GET /refund?orderId=...` 呼叫後直接建立退款申請	GET 依 HTTP 語意應是安全、可快取的讀取操作，不該有副作用	瀏覽器預載、爬蟲、CDN、監控探針或網路重試都可能自動重發 GET，造成誤觸或重複建立退款申請

另外要指出觀念錯誤：CORS 只約束瀏覽器的跨來源行為，擋不住直接對 API 發出的請求（curl、Postman、另一台伺服器），所以「CORS 僅允許官網來源」不能當作授權依據。業務規則「只有訂單的原購買人可以申請自助退款」必須在後端以「登入身分 vs 訂單 purchaserId」比對落實；建立退款應改為 `POST /orders/{orderId}/refunds` 之類的寫入方法。

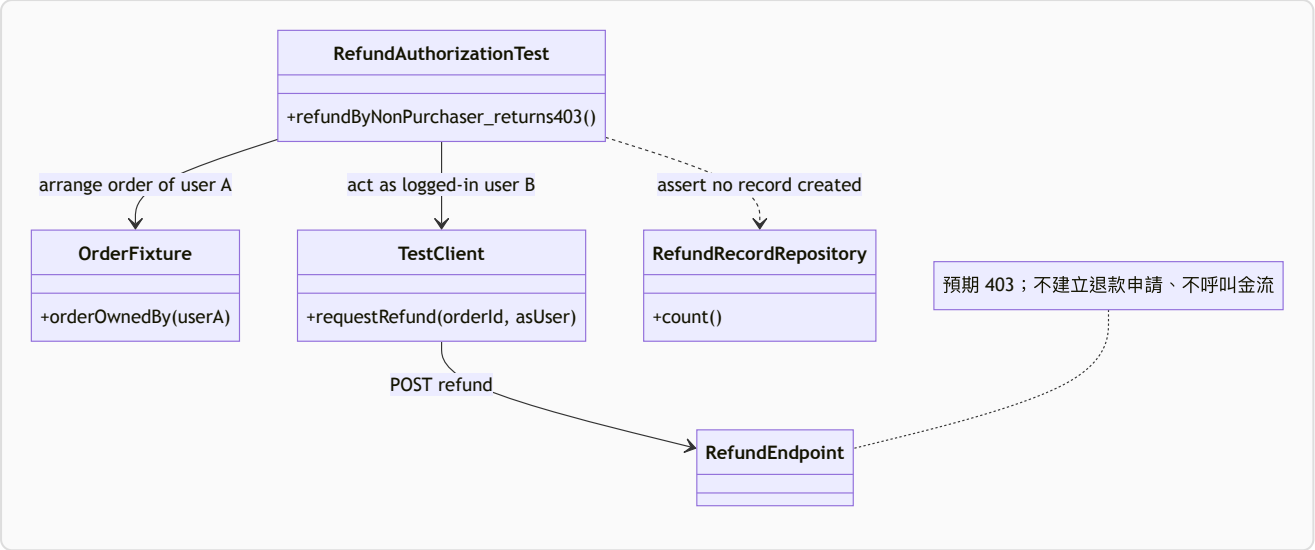


第 2 題：會補哪個測試？預期得到什麼結果？

現有測試只有一個 mock 成功案例，沒有涵蓋業務規則的拒絕路徑。我會優先補「非原購買人被拒絕」的授權測試：

測試案例	前置	操作	預期結果
非原購買人申請退款	訂單屬於使用者 A；使用者 B 已登入	B 對 A 的訂單送出退款申請	回 `403 Forbidden`；不建立退款申請、不呼叫金流
未登入請求	無有效 session	直接呼叫退款 API	回 `401 Unauthorized`
原購買人申請（對照組）	A 已登入，訂單屬於 A 且符合退款資格	A 送出申請	受理並進入退款流程

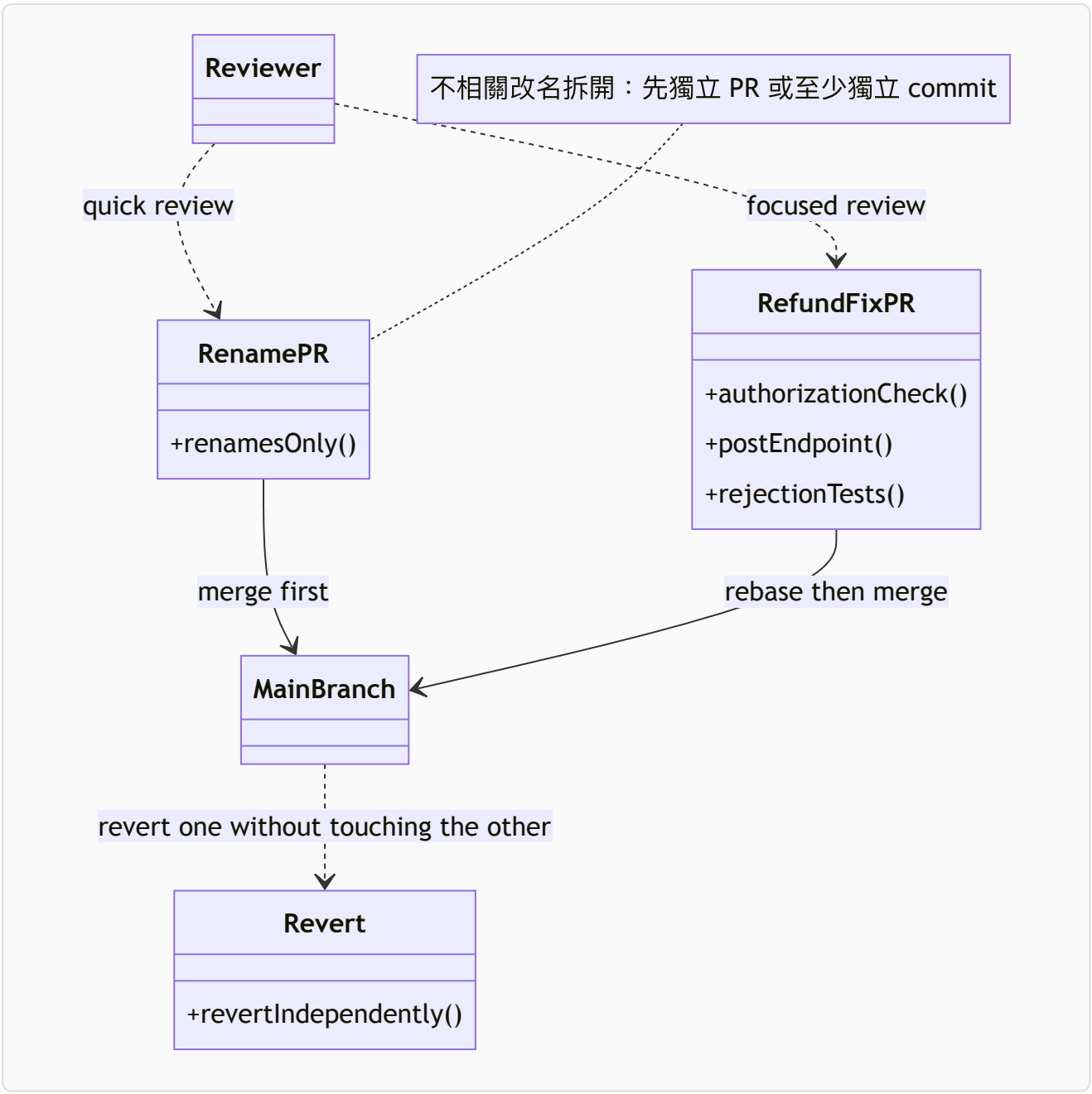
第一個案例最關鍵：它直接驗證本題業務規則「只有原購買人可以申請」，也是這份 PR 漏掉的路徑。測試應打在真實的授權邏輯上，而不是 mock 掉授權本身。



第 3 題：不相關的檔名修改怎麼安排？怎麼讓變更好審查、好回退？

安排	做法	好處
拆成獨立 PR	純檔名修改先送一個 PR（不含行為改變），退款修正另一個 PR，後者 rebase 到前者合併後的 main	改名 PR 可快速看過；退款 PR 的 diff 只剩業務相關改動，審查聚焦
若暫時維持同一 PR	至少分成獨立 commit：一個 commit 純改名、一個 commit 功能修改，並在 PR 描述說明順序	審查者可逐 commit 檢視，不會混在一起
出問題時的回退	兩個變更各自獨立 revert	回退退款功能不會把改名一起倒回去，反之亦然

原則是一個 PR 只做一件事。不相關的改名會放大 diff、稀釋審查注意力，也會讓回退變成綁在一起的全有或全無。


Reviewer
不相關改名拆開：先獨立 PR 或至少獨立 commit
quick review
focused review
RenamePR
+renamesOnly()
RefundFixPR
+authorizationCheck()
+postEndpoint()
+rejectionTests()
merge first
rebase then merge
MainBranch
revert one without touching the other
Revert
+revertIndependently()

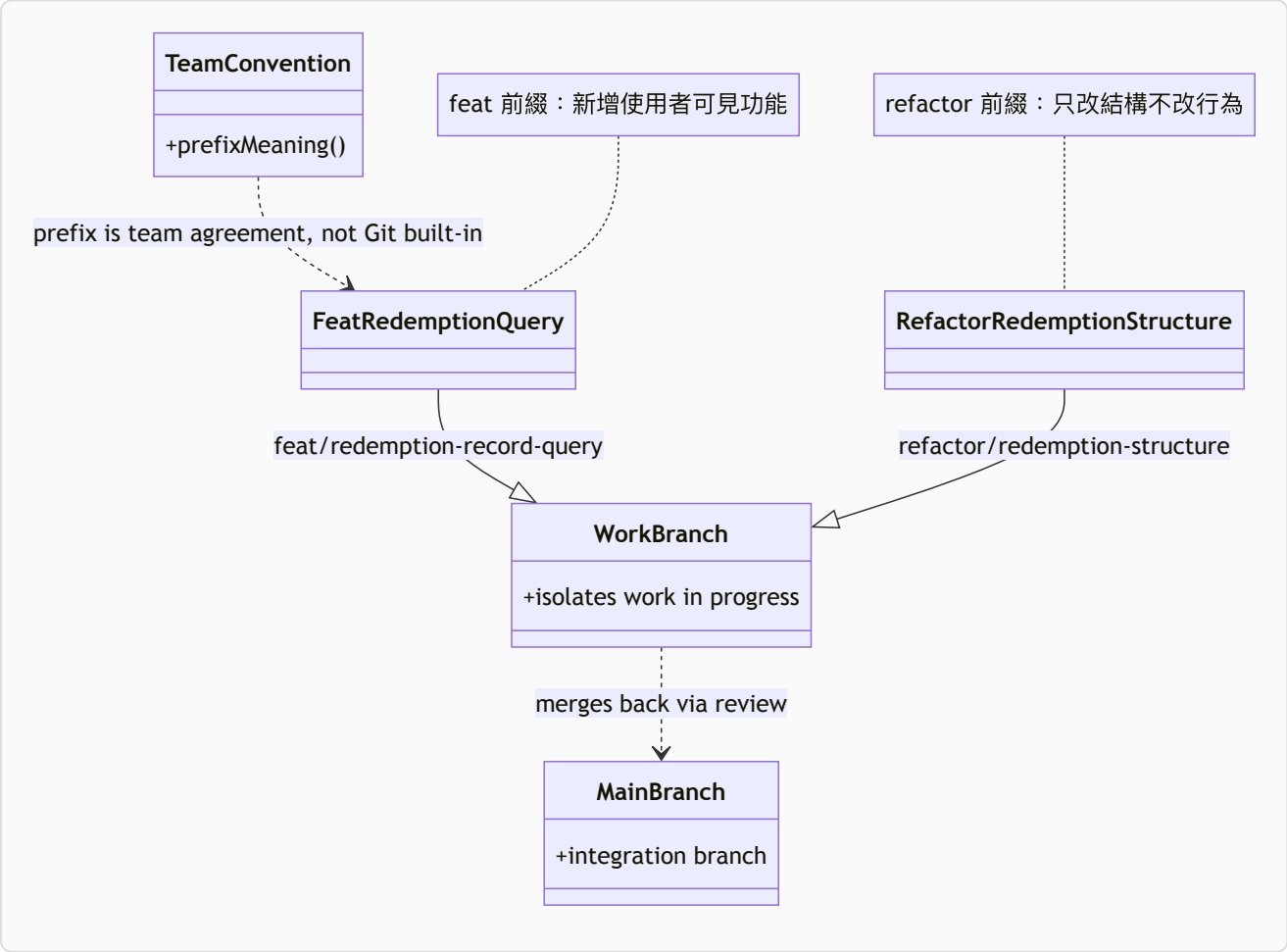
情境七｜從工作分支，到一起交付

第 1 題：工作分支的用途、分支命名，以及前綴是 Git 內建還是團隊約定？

工作分支的用途是把進行中的修改與 main 隔離：main 保持可整合、可交付的狀態；一項工作一個分支，方便獨立開發、測試、審查與回退。

工作	分支名稱	前綴想傳達
新增贈品核銷紀錄查詢	`feat/redemption-record-query`	feat：新增使用者可見的功能，審查重點在行為是否正確
整理既有核銷程式、不改變行為	`refactor/redemption-structure`	refactor：只改結構、不改行為，審查不用找行為差異，重點在可讀性與測試仍全綠

這些前綴是**團隊約定**，不是 Git 的內建功能。Git 只把分支名當字串、把斜線當路徑分隔；前綴的語意來自團隊規範（常搭配 conventional commits 或分支流程文件），讓人從名稱就能判斷工作性質與審查重點。

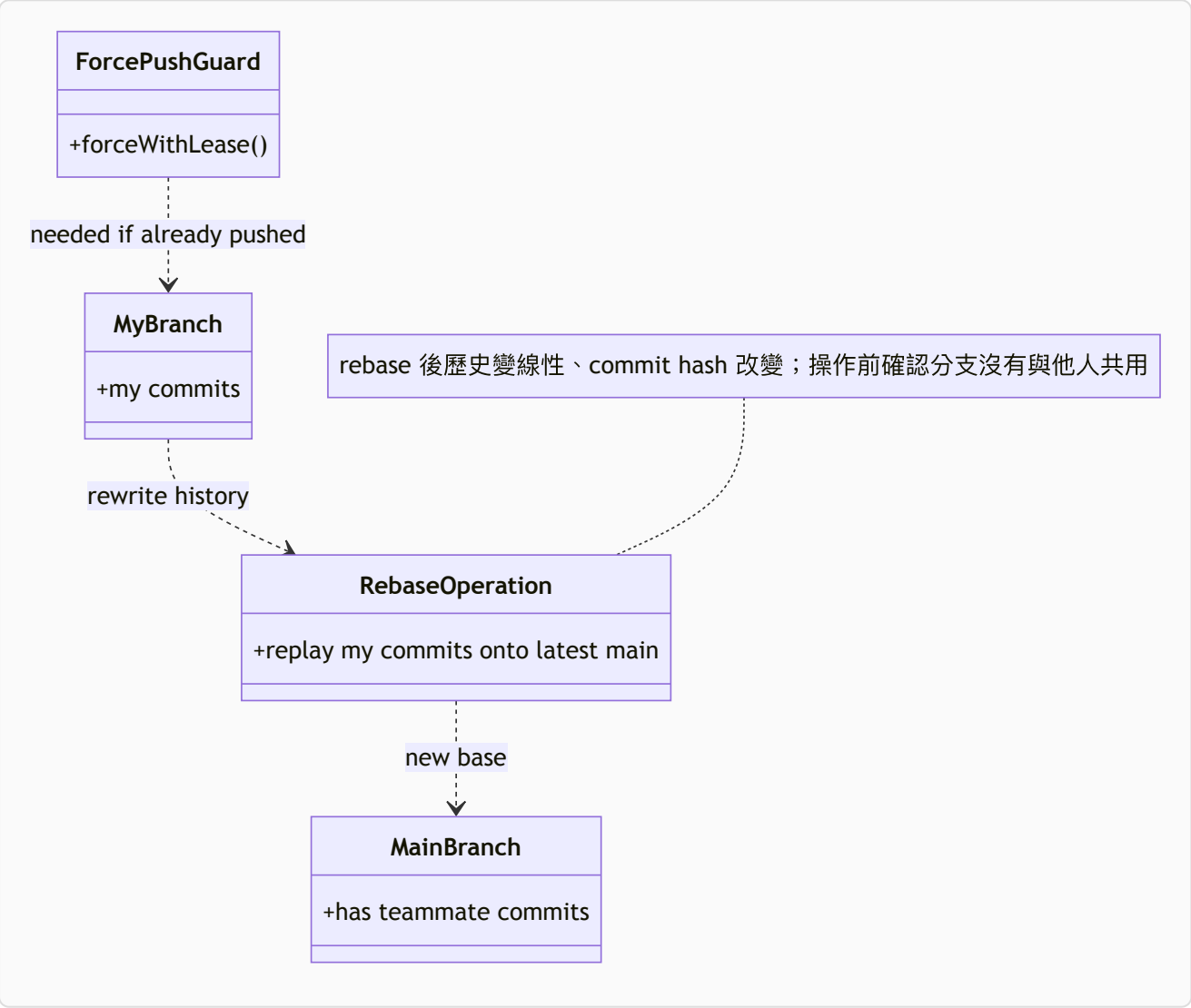


第 2 題：把工作分支 rebase 到最新 main 是在做什麼？前後差異與一個注意事項

rebase 是把自己分支上的 commit「拔下來，逐一重播到最新 main 的尾端」，讓歷史看起來像是從最新 main 直接長出來的；重播過程中若與同事的修改衝突，就逐個 commit 解決。

面向	rebase 前	rebase 後
分支起點	從較舊的 main 分岔	改接到最新 main 的尾端
與同事的修改	分叉並行，落後 main 上的新 commit	我的 commit 排在同事的 commit 之後，呈線性
commit hash	原本的值	全部改變（歷史被重寫）

操作前要留意的一件事：rebase 會重寫歷史。如果分支已經 push 過，要確認只有自己在使用它（沒有同事基於它開發），重 push 時用 `--force-with-lease` 而非強制覆蓋；操作前也確認工作區乾淨，免得衝突解到一半汙染未提交的修改。

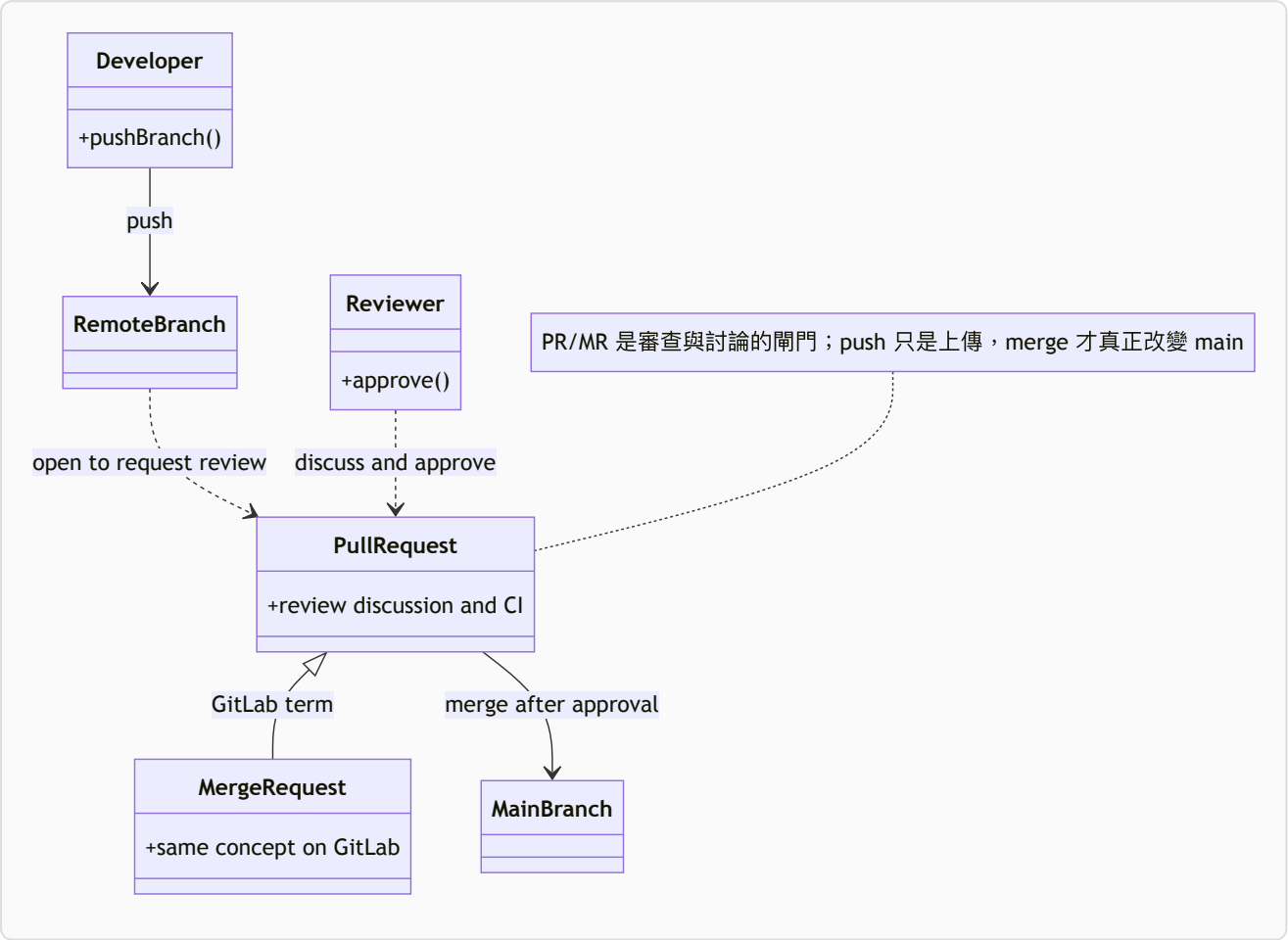


第 3 題：PR 與 MR 各指什麼？push、建立 PR/MR、合併進 main 有什麼不同？

PR 是 Pull Request（GitHub、Bitbucket 用語），MR 是 Merge Request（GitLab 用語），本質是同一件事：請求把某個分支合併進目標分支的協作單元。它們在流程中扮演「程式碼進 main 之前的閘

門」，承載審查留言、討論、核准紀錄與 CI 檢查結果。

動作	本質	在流程中的角色
push 工作分支	把本地 commit 上傳到遠端	只是備份與分享，沒有人被要求審查，main 不受影響
建立 PR / MR	正式提出「請把我的分支合併進 main」	開啟審查與討論，觸發 CI；此時 main 仍未改變
合併進 main	審查通過後執行	真正改變整合分支，之後所有人的工作都以此為基礎



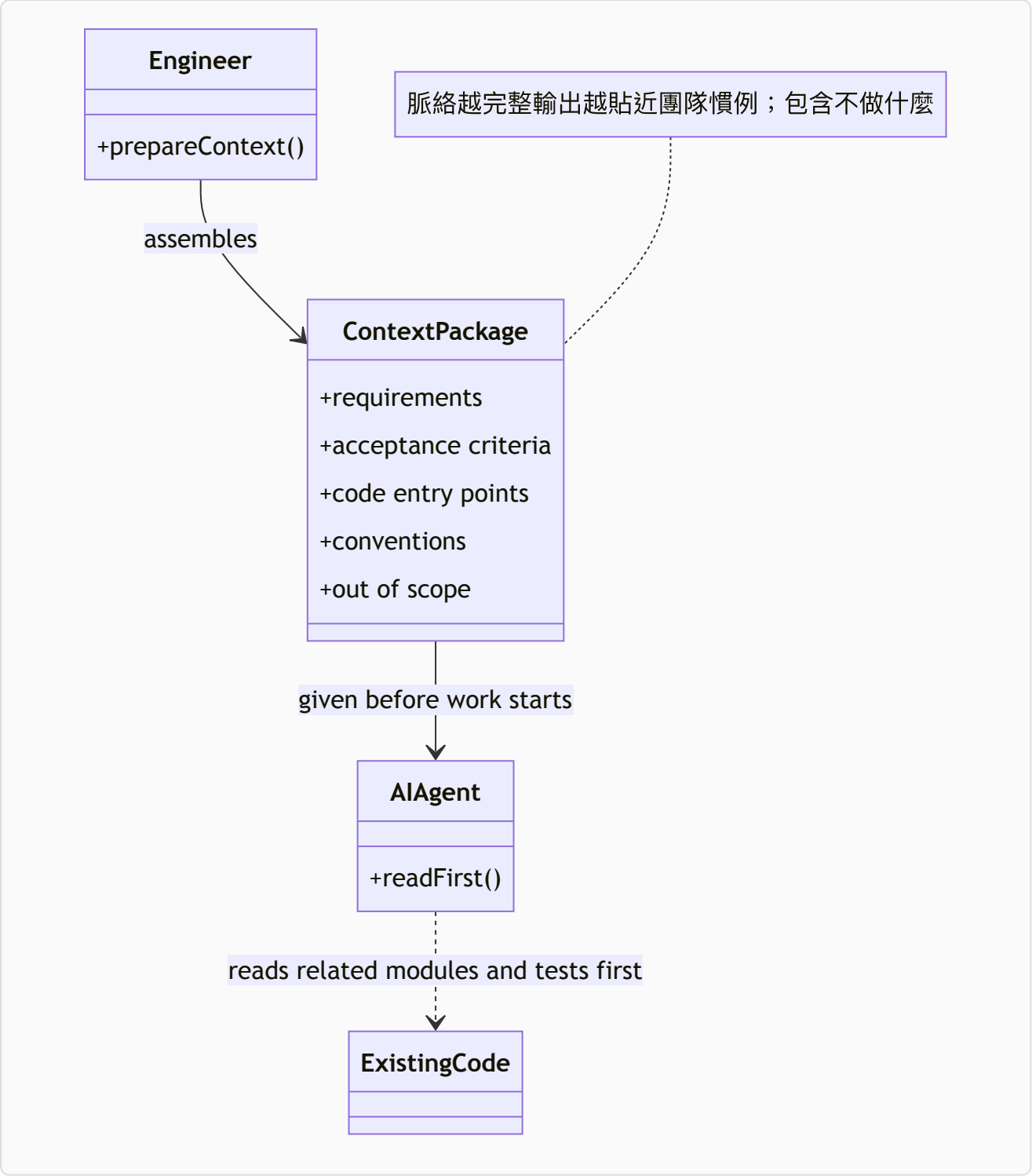
情境八｜當 AI 成為搭檔，你會怎麼開始？

以下以新增「贈品核銷紀錄查詢」為例，說明我與 AI Agent 的分工安排。

第 1 題：開始之前，會先給 Agent 哪些資訊？為什麼？

給 Agent 的資訊	內容	為什麼
需求與驗收	PM 的原話、題目限制、「完成」的定義	讓它對齊目標，不自己猜需求
程式脈絡	相關模組的入口、負責檔案、既有測試，請它先讀並回述理解	Agent 的輸出品質取決於脈絡；先讀再動手才能沿用既有慣例
團隊慣例	架構分層（如 use case / port / adapter）、命名、分支與提交格式	讓輸出風格一致、能直接進 code review
邊界	哪些檔案不能動；規格外的決定要標「待確認」而不是自行決定	防止夾帶無關修改與未經核准的假設

理由是：先對齊「做什麼、照什麼慣例、不動什麼」，可以省去大量來回修正，也讓後面的審查有依據。

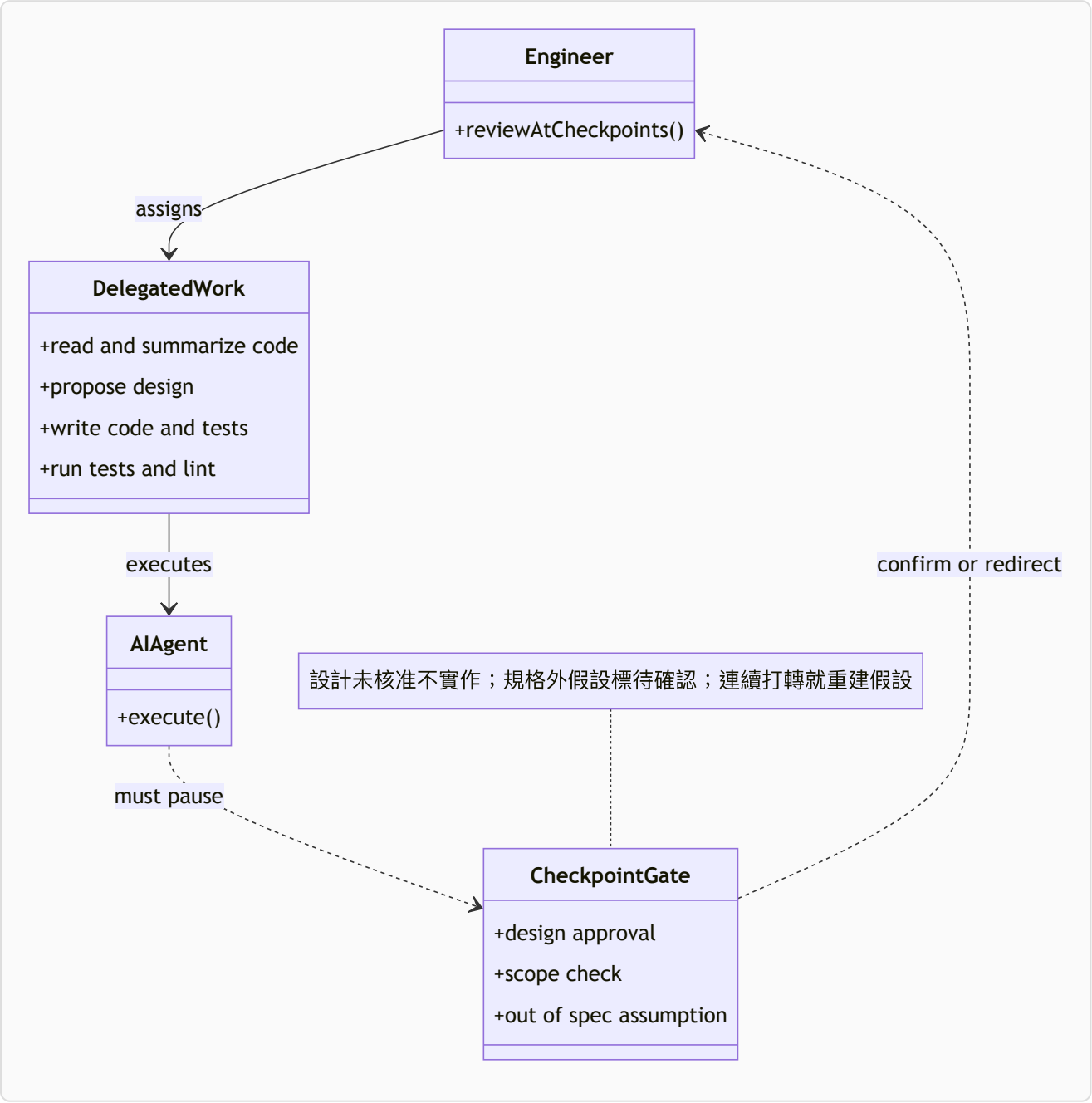


第 2 題：哪些工作交給它？什麼時候停下來確認或調整方向？

交給 Agent	我保留 / 停下確認的時機
讀程式、整理現況與影響範圍	它的理解與事實不符時，先對齊再動手
依核准的設計寫初版程式與測試	設計提案未核准前，不進入實作（plan gate）

交給 Agent	我保留 / 停下確認的時機
跑測試、lint、整理 PR 描述	它做出規格外假設、改到無關模組、或想放寬測試讓紅燈變綠時，停下來人工判斷

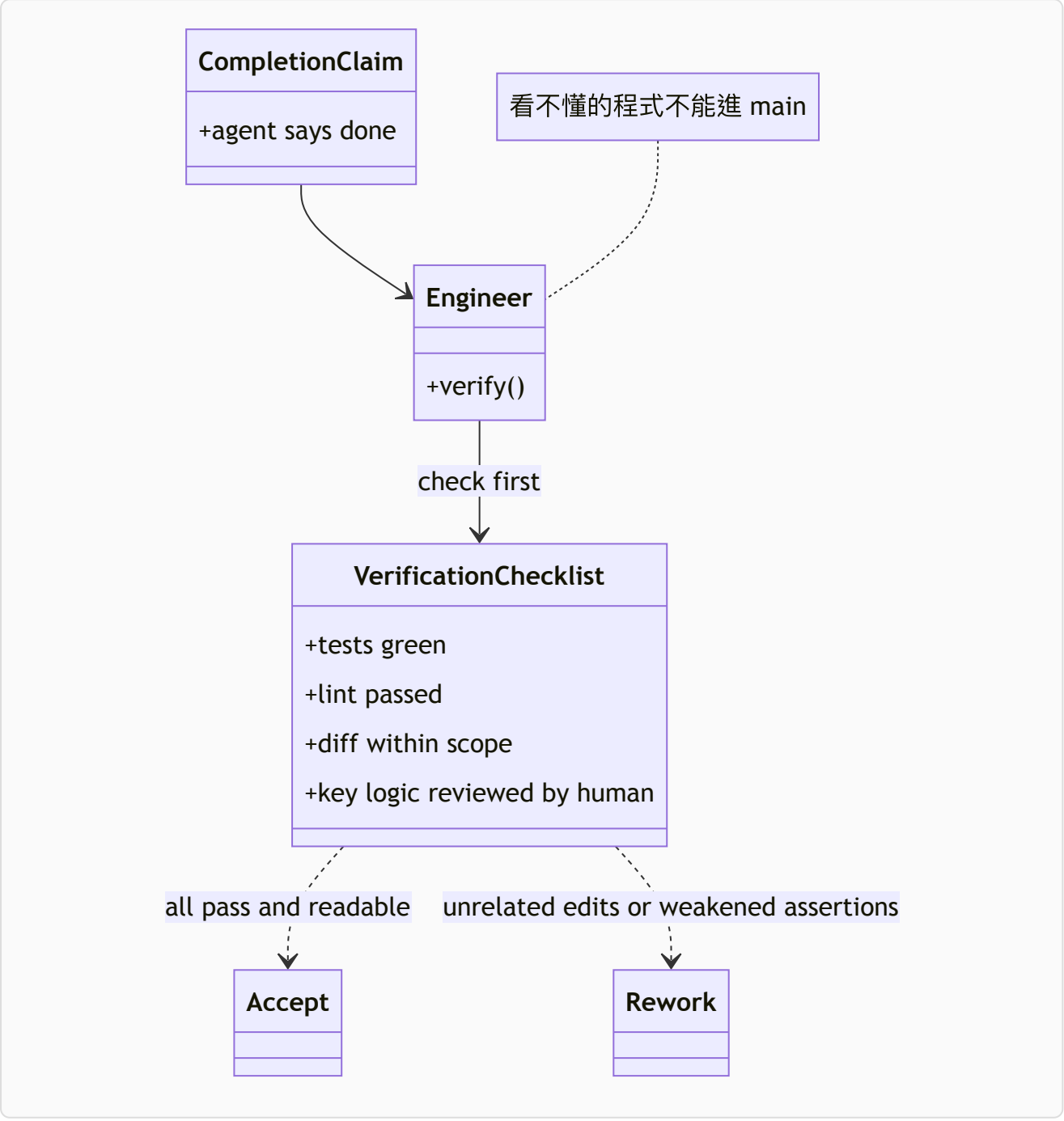
調整方向的訊號：如果它在同一個問題上連續打轉（同類錯誤反覆出現），我會停下來回到錯誤訊息與程式本身，形成具體假設後換個指示或縮小任務範圍，而不是讓它無限重試。



第 3 題：Agent 說「完成了」，你會先看什麼？什麼情況接受、什麼情況請它再改？

先看什麼	判斷
驗證證據：測試是否全綠、lint 是否通過、實際跑了哪些指令	沒有證據不算完成
diff 範圍：是否只動了該動的檔案、有沒有夾帶無關修改	範圍正確才進下一步
關鍵邏輯親自讀一遍（金流、授權、併發等不能只看測試的路徑）	看不懂的程式不能進 main

接受的情況：範圍正確、驗證通過、關鍵路徑我讀得懂且同意設計、PR 描述完整。請它再改的情況：夾帶無關修改、為了過測試而放寬斷言、出現沒討論過的設計決策，或產出我解釋不了的程式。

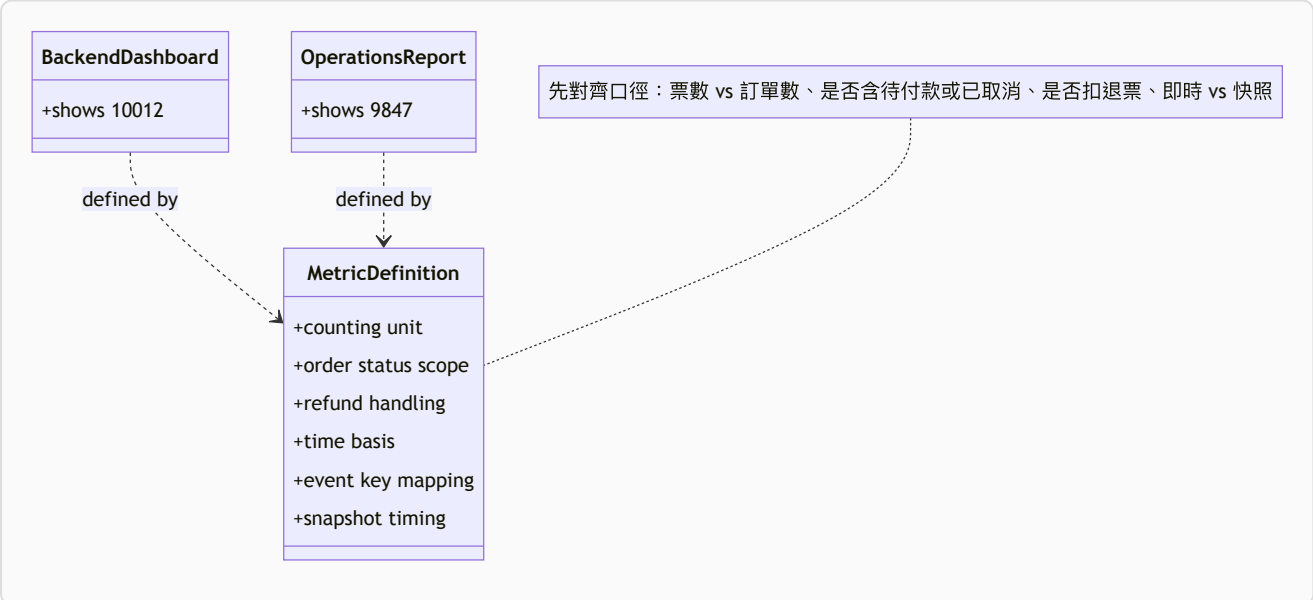


情境九｜數字對不上，差異從哪裡來？

第 1 題：先確認哪些定義？哪些差異可能只是「算的東西不一樣」？

定義面向	可能的口徑差異	怎麼確認
計數單位	票券張數 vs 訂單筆數（一筆訂單可含多張票）	看兩邊查詢數的是 tickets 還是 orders
訂單狀態	只算已付款，或把待付款、已取消也算進去	檢查兩邊查詢的 where 條件
退款處理	是否扣除已退票的票券	檢查是否過濾已退款票券
時間口徑	以購買時間、付款時間或場次結束時間切	對齊使用的時間欄位
場次對應	PostgreSQL 的 eventId 與 MongoDB 場次設定的鍵是否一致	比對兩邊用來對應場次的鍵值
更新時機	後台即時查詢 vs 報表離線快照（例如 T+1）	問報表的產製排程與資料截止時間

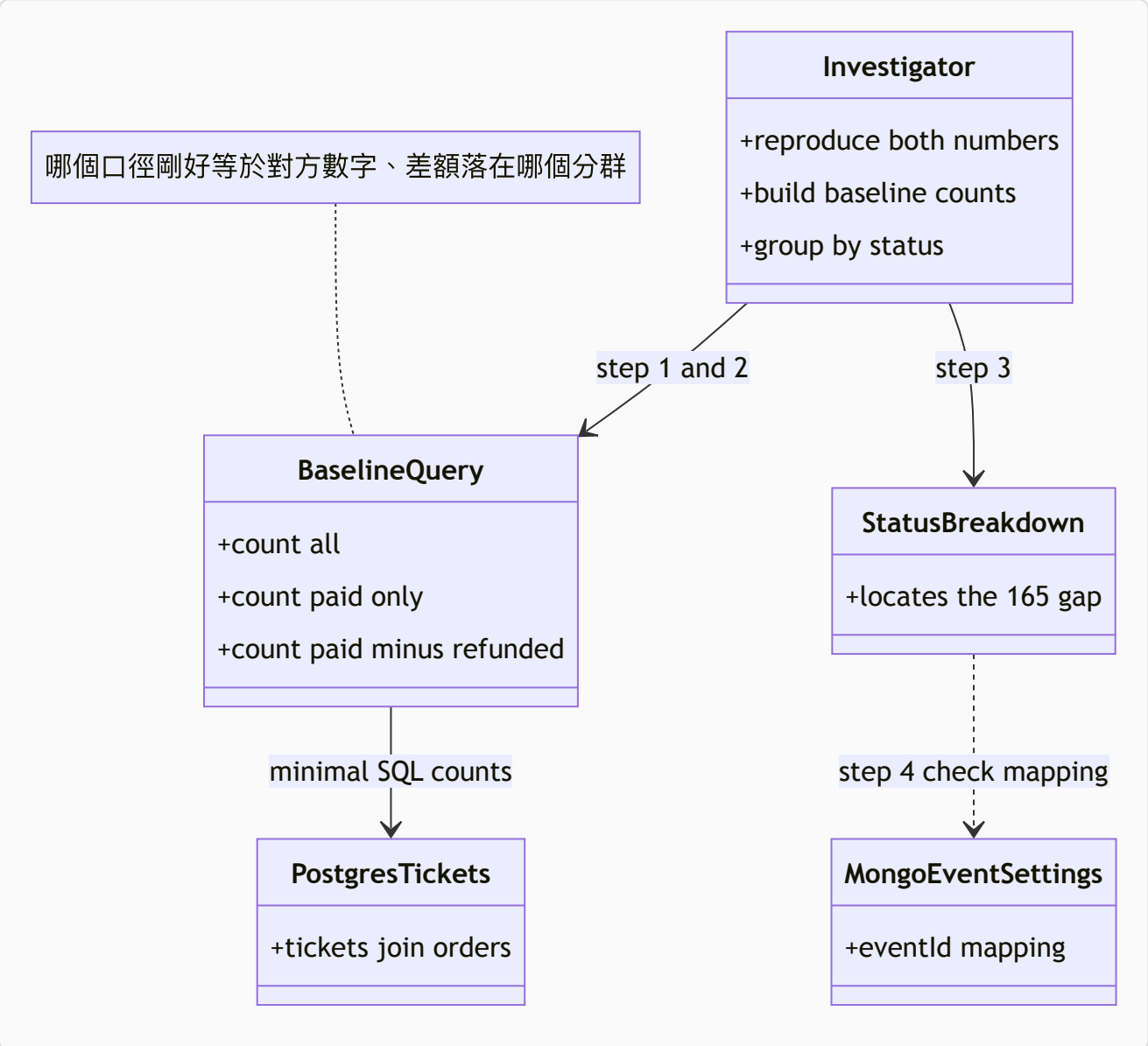
以上每一項都可能只是「雙方算的東西不一樣」：例如後台即時數含待付款訂單的票、報表快照只算已付款，就會出現這種差距，不代表資料壞了。先對齊定義再查資料，才不會把口徑差當成 bug 追。



第 2 題：怎麼一步一步縮小問題？查法與順序

步驟	做法	目的
1. 重現兩個數字	找到後台與報表各自的查詢（或程式碼），用同一場次重跑，確認穩定得到 10,012 與 9,847	確認問題可重現，不是暫時性或看錯場次
2. 建立基準計數	直接對 PostgreSQL 下最小 SQL，各口徑分別 count	看哪個口徑剛好等於哪個數字
3. 差額分群	依訂單狀態、是否退票 GROUP BY，找出 165 張差額落在哪個分群	把差異定位到最小範圍
4. 查資料異常	口徑對齊後仍不一致，才查重複票、孤兒票（訂單不存在）、狀態矛盾、MongoDB 場次鍵對應錯誤	確認是否資料本身真的壞了

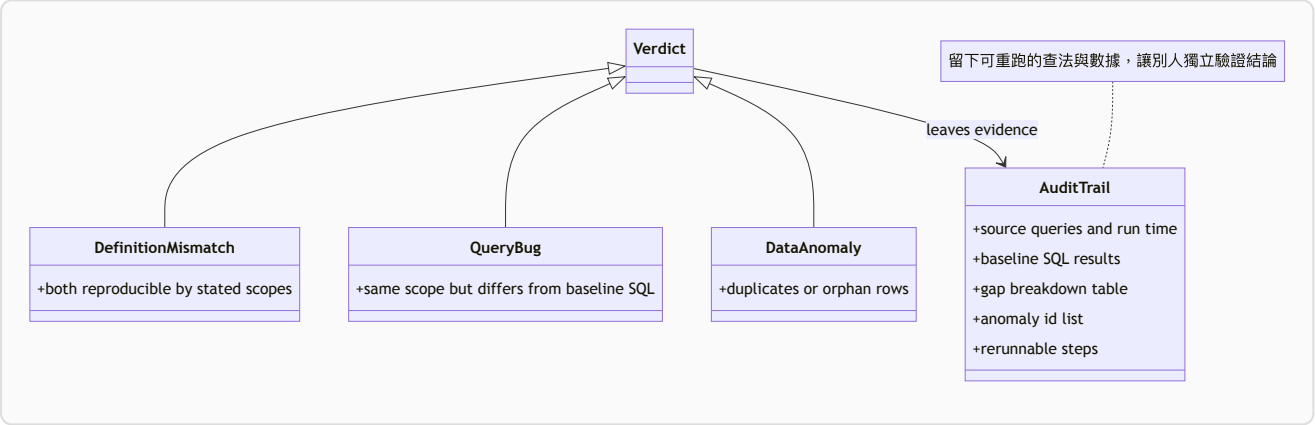
第 2 步的基準 SQL 例如：`SELECT count(*) FROM tickets t JOIN orders o ON t.order_id = o.id WHERE o.event_id = :eid`（全部）、加上 `AND o.status = 'paid'`（只算已付款）、再加 `AND t.refunded_at IS NULL`（扣掉已退票）。如果 10,012 是「全部」、9,847 是「已付款且未退票」，差額 165 又能被狀態分群完整解釋，就是口徑問題；對不上才往資料異常查。



第 3 題：怎麼判斷是定義不同、查詢問題還是資料異常？留下哪些資訊？

判斷	特徵	後續處理
統計定義不同	兩個數字都能用明確口徑重現，且口徑差恰好解釋差額（例如 165 恰好是待付款＋已取消訂單的票數）	對齊口徑、把定義寫進報表說明，不改資料
查詢邏輯問題	同一口徑下，報表查詢的結果與手寫基準 SQL 不一致（join 遺漏、條件寫錯、快照過期）	修查詢或調整快照排程
資料異常	基準 SQL 也對不上，且能列出具體異常資料列（重複票、孤兒票、狀態矛盾如已取消卻有有效票）	走資料修復流程，並記錄修復依據

會留下的資訊：兩邊數字的來源查詢與執行時間；我用的基準 SQL、各口徑的 count 結果與差額分群表；最終結論與判斷依據；若有資料異常，附上異常票的 id 清單。所有查法都可重跑，讓別人能獨立驗證同樣的結論。



延伸分享｜有沒有一次，你改變了與 AI 合作的方式？

這題是自由分享，可以略過或留到面談。若要簡短回答，方向例如：一開始把整個需求一次丟給 AI，結果它自行補了規格外的假設、產出很難審的大 diff；後來改成先給足脈絡、設計先核准再分段實作，每段都要求測試與 lint 的證據，來回次數與返工大幅降低——也就是情境八安排的那種合作方式。